Trabajo Práctico Final: Estrategia de Testing (Aplicación Clínica)

Integrantes: Ganduglia, Ciro. Rosal, Fernando Daniel. Sosa, Lucas. Stufano, Nazareno.

---

1. Presentación y Objetivos

El presente trabajo práctico tiene como propósito demostrar la adquisición de las capacidades fundamentales para el testing de un producto de software. El desarrollo exige estructurar las validaciones diferenciando estrictamente dos niveles arquitectónicos:

- **Testing atómico (unitario):** sobre componentes independientes en memoria sin dependencias externas.
- **Testing de integración:** sobre componentes acoplados con recursos externos (motor relacional de base de datos MySQL y concurrencia multihilo).

---

2. Consignas y Desarrollo

Consigna 1: Testing de la Clase AsociadoDTO

1. Escenarios de Prueba

- **Escenario E1:** Instancia de AsociadoDTO recién creada con el constructor por defecto sin inicializar datos.
- **Escenario E2:** Instancia de AsociadoDTO en memoria con datos cargados y atributos válidos asignados.

2. Tabla de Particiones de Equivalencia

Atributo / Variable	Clase Válida	Clase Inválida
nombre	nombre != null → Clase 1	nombre == null → Clase 2
apellido	apellido != null → Clase 3	apellido == null → Clase 4
dni	dni != null → Clase 5	dni == null → Clase 6
ciudad	ciudad != null → Clase 7	ciudad == null → Clase 8
calle	calle != null → Clase 9	calle == null → Clase 10
numero	numero >= 0 → Clase 11	numero < 0 → Clase 12
telefono	telefono != null → Clase 13	telefono == null → Clase 14

3. Batería de Pruebas

Categoría	Casos de Prueba / Entradas	Resultado Esperado	Clases Cubiertas	Escenario
Clases Correctas	nombre="Juan", apellido="Pérez", dni="12345678", ciudad="MdP", calle="Jujuy", numero=100, telefono="2231234567"	Asignación exitosa de todos los atributos y recuperación idéntica en getters.	1, 3, 5, 7, 9, 11, 13	E1 → E2
Clases Correctas	numero = 0 (Valor límite inferior según SRS), resto de campos válidos.	Asignación exitosa del valor 0. (Detecta fallo si la implementación exige numero > 0).	11 (Valor límite 0)	E2
Clases Incorrectas	nombre = null, resto de campos válidos.	Lanza AssertionError.	2	E1 / E2
Clases Incorrectas	apellido = null, resto de campos válidos.	Lanza AssertionError.	4	E1 / E2

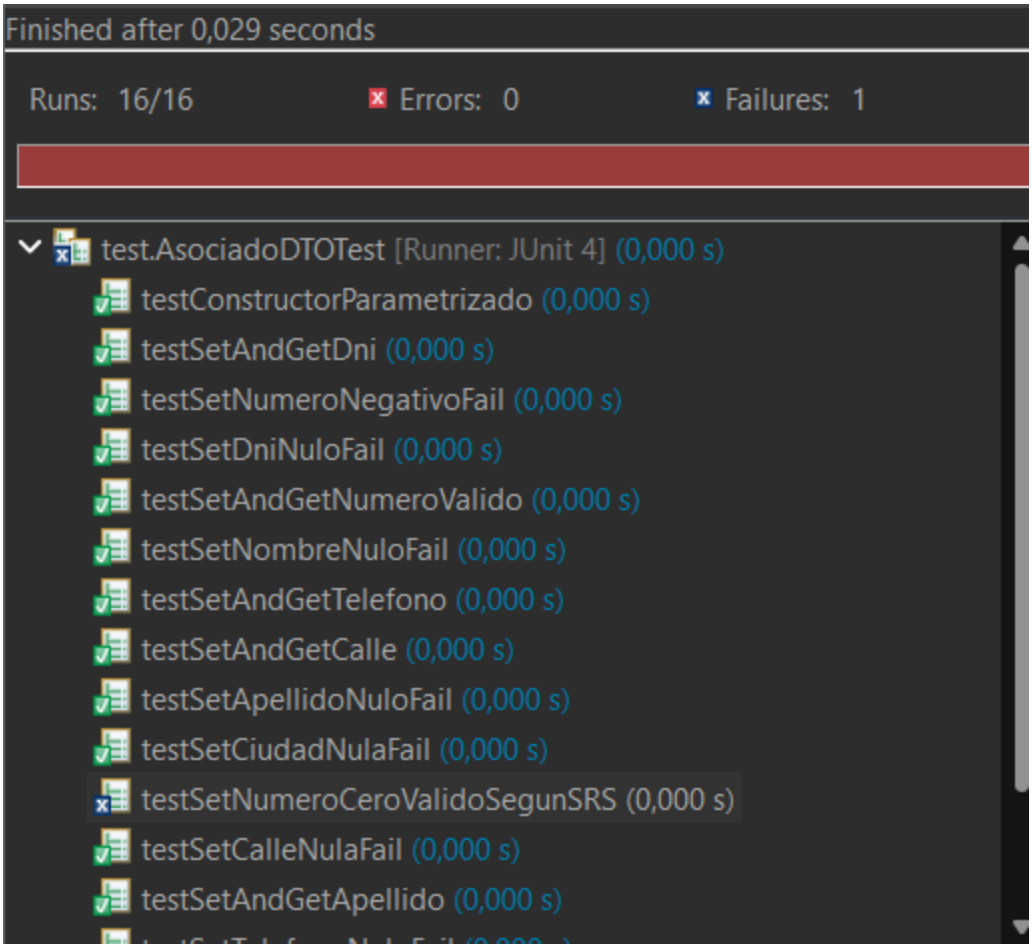
Categoría	Casos de Prueba / Entradas	Resultado Esperado	Clases Cubiertas	Escenario
Clases Incorrectas	dni = null, resto de campos válidos.	Lanza AssertionError.	6	E1 / E2
Clases Incorrectas	ciudad = null, resto de campos válidos.	Lanza AssertionError.	8	E1 / E2
Clases Incorrectas	calle = null, resto de campos válidos.	Lanza AssertionError.	10	E1 / E2
Clases Incorrectas	numero = -1 (Valor fuera de rango), resto de campos válidos.	Lanza AssertionError.	12	E1 / E2
Clases Incorrectas	telefono = null, resto de campos válidos.	Lanza AssertionError.	14	E1 / E2

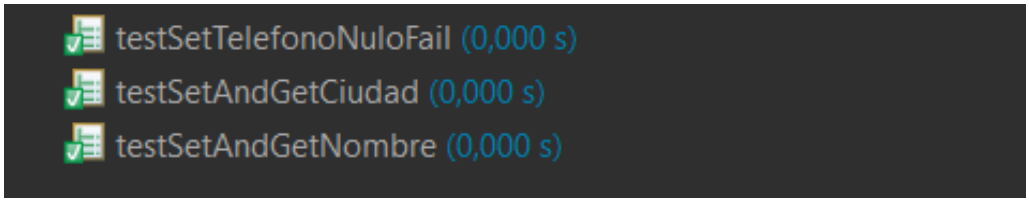
Apéndice A: Matriz de Casos de Prueba (AsociadoDTOTest)

ID	Método de Prueba	Entradas / Precondiciones	Resultado Esperado	Propósito / Verificación
TC01	testSetAndGetNombre	nombre = "Juan"	Retorna "Juan"	Verificación de getter/setter de nombre.
TC02	testSetAndGetApellido	apellido = "Pérez"	Retorna "Pérez"	Verificación de getter/setter de apellido.
TC03	testSetAndGetDni	dni = "12345678"	Retorna "12345678"	Verificación de getter/setter de dni.
TC04	testSetAndGetCiudad	ciudad = "Mar del Plata"	Retorna "Mar del Plata"	Verificación de getter/setter de ciudad.
TC05	testSetAndGetCalle	calle = "San Martín"	Retorna "San Martín"	Verificación de getter/setter de calle.
TC06	testSetAndGetNumeroValido	numero = 1234	Retorna 1234	Verificación de asignación de número positivo.

ID	Método de Prueba	Entradas / Precondiciones	Resultado Esperado	Propósito / Verificación
TC07	testSetAndGetTelefono	telefono = "2231234567"	Retorna "2231234567"	Verificación de getter/setter de telefono.
TC08	testConstructorParametrizado	Todos los campos válidos	Estado interno coincidente	Verificación del constructor con parámetros.
TC09	testSetNombreNuloFail	nombre = null	Lanza AssertionError	Verificación de control contra valores nulos.
TC10	testSetApellidoNuloFail	apellido = null	Lanza AssertionError	Verificación de control contra valores nulos.
TC11	testSetDniNuloFail	dni = null	Lanza AssertionError	Verificación de control contra valores nulos.
TC12	testSetNumeroNegativoFail	numero = -5	Lanza AssertionError	Rechazo de valores enteros negativos.

ID	Método de Prueba	Entradas / Precondiciones	Resultado Esperado	Propósito / Verificación
TC13	testSetNumeroCeroValidoSegunSRS	numero = 0	Asignación exitosa (0)	Validación del valor límite 0 según especificación SRS (SRS 3.1).





Conclusiones Consigna 1

1. **Aislamiento del Test:** La suite AsociadoDTOTest evalúa la entidad AsociadoDTO en memoria sin requerir la instanciación de servicios externos, conexiones a MySQL ni capas de interfaz gráfica.
2. **Sensibilidad a la Configuración de la JVM:** La validación mediante la sentencia assert requiere explícitamente la bandera -ea en el entorno de ejecución para actuar de forma efectiva. En entornos productivos donde las aserciones suelen desactivarse, es recomendable sustituir las aserciones nativas por excepciones explícitas (IllegalArgumentException).
3. **Consistencia con los Requisitos (SRS):** Se detectó que el setter de numero evalúa numero > 0, mientras que la especificación SRS establece que el valor debe ser mayor o igual a cero (numero >= 0). Se sugiere ajustar la condición a assert numero >= 0; antes de avanzar al Inciso 2.

Código Fuente Completo: AsociadoDTOTest.java

```
package test;

import static org.junit.Assert.*;
import org.junit.Before;
import org.junit.Test;
import persistencia.AsociadoDTO;

public class AsociadoDTOTest {

    private AsociadoDTO asociado;

    @Before
```

```
    public void setUp() {
        asociado = new AsociadoDTO();
    }

    @Test
    public void testSetAndGetNombre() {
        asociado.setNombre("Juan");
        assertEquals("El nombre recuperado debe ser igual al asignado", "Juan", asociado.getNombre());
    }

    @Test
    public void testSetAndGetApellido() {
        asociado.setApellido("Pérez");
        assertEquals("El apellido recuperado debe ser igual al asignado", "Pérez", asociado.getApellido());
    }

    @Test
    public void testSetAndGetDni() {
        asociado.setDni("12345678");
        assertEquals("El DNI recuperado debe ser igual al asignado", "12345678", asociado.getDni());
    }

    @Test
    public void testSetAndGetCiudad() {
        asociado.setCiudad("Mar del Plata");
        assertEquals("La ciudad recuperada debe ser igual a la asignada", "Mar del Plata", asociado.getCiudad());
    }

    @Test
    public void testSetAndGetCalle() {
        asociado.setCalle("San Martín");
        assertEquals("La calle recuperada debe ser igual a la asignada", "San Martín", asociado.getCalle());
    }

    @Test
    public void testSetAndGetNumeroValido() {
        asociado.setNumero(1234);
```

```
        assertEquals("El número recuperado debe ser igual al asignado", 1234, asociado.getNumero());
    }

    @Test
    public void testSetAndGetTelefono() {
        asociado.setTelefono("2231234567");
        assertEquals("El teléfono recuperado debe ser igual al asignado", "2231234567", asociado.getTelefono());
    }

    @Test
    public void testConstructorParametrizado() {
        AsociadoDTO dto = new AsociadoDTO("Ana", "Gómez", "87654321", "Mar del Plata", "Belgrano", 456,
"2239876543");
        assertEquals("Ana", dto.getNombre());
        assertEquals("Gómez", dto.getApellido());
        assertEquals("87654321", dto.getDni());
        assertEquals("Mar del Plata", dto.getCiudad());
        assertEquals("Belgrano", dto.getCalle());
        assertEquals(456, dto.getNumero());
        assertEquals("2239876543", dto.getTelefono());
    }

    @Test(expected = AssertionError.class)
    public void testSetNombreNuloFail() {
        asociado.setNombre(null);
    }

    @Test(expected = AssertionError.class)
    public void testSetApellidoNuloFail() {
        asociado.setApellido(null);
    }

    @Test(expected = AssertionError.class)
    public void testSetDniNuloFail() {
        asociado.setDni(null);
    }
```

```
    @Test(expected = AssertionError.class)
    public void testSetCiudadNulaFail() {
        asociado.setCiudad(null);
    }

    @Test(expected = AssertionError.class)
    public void testSetCalleNulaFail() {
        asociado.setCalle(null);
    }

    @Test(expected = AssertionError.class)
    public void testSetTelefonoNuloFail() {
        asociado.setTelefono(null);
    }

    @Test(expected = AssertionError.class)
    public void testSetNumeroNegativoFail() {
        asociado.setNumero(-5);
    }

    @Test
    public void testSetNumeroCeroValidoSegunSRS() {
        asociado.setNumero(0);
        assertEquals("El número 0 debería ser aceptado según la SRS", 0, asociado.getNumero());
    }
}
```

Consigna 2: Testing de Integración (AsociadoDTO, DataAccessObject y BaseDeDatos)

1. Análisis de Código y Detección de Inconsistencias

- **Gestión de Conexión en BaseDeDatos:** La URL configurada es jdbc:mysql://localhost:3306/?useSSL=false&serverTimezone=UTC. El método iniciarBD() crea la base de datos Grupo_7 si no existe, ejecuta USE Grupo_7;, elimina la tabla

Asociados (DROP TABLE IF EXISTS) y la recrea poblándola con 20 datos semilla.

- **Mapeo y Consultas SQL en DataAccessObject:** La consulta INSERT utiliza DNI en mayúsculas (INSERT INTO Asociados(nombre, apellido, DNI, calle, numero, ciudad, telefono)), mientras que eliminar utiliza WHERE dni = ?. El método CrearDTO mapea los campos desde el ResultSet.
- **Inconsistencias y Reglas de Integridad:** La columna DNI posee restricción UNIQUE en MySQL. Insertar un DNI duplicado dispara java.sql.SQLIntegrityConstraintViolationException.

2. Escenarios y Particiones de Equivalencia

- **Escenario E1:** Motor MySQL activo en localhost:3306 con la BD Grupo_7 inicializada mediante iniciarBD() (contiene los 20 asociados semilla).
- **Escenario E2:** Motor MySQL activo sin haber ejecutado iniciarBD() previamente o con tablas inexistentes.

Componente / Operación	Clase Válida	Clase Inválida
Patrón Singleton (BaseDeDatos)	Instancia única compartida → Clase 1	Creación de múltiples instancias o referencias nulas → Clase 2
Conexión SQL (getConexion)	Estado activo y conexo a MySQL → Clase 3	Credenciales incorrectas / Puerto cerrado / BD no seleccionada → Clase 4
Inserción de Asociado (guardar)	Objeto Asociado válido con DNI único → Clase 5	Asociado == null → Clase 6.1 / DNI duplicado → Clase 6.2

Componente / Operación	Clase Válida	Clase Inválida
Consulta con Límite (cargarConLimite)	limite > 0 → Clase 7	limite <= 0 → Clase 8
Búsqueda por ID (cargar)	id > 0 existente en BD → Clase 9	id <= 0 → Clase 10.1 / ID inexistente → Clase 10.2
Eliminación por DNI (eliminar)	dni != null existente → Clase 11	dni == null → Clase 12

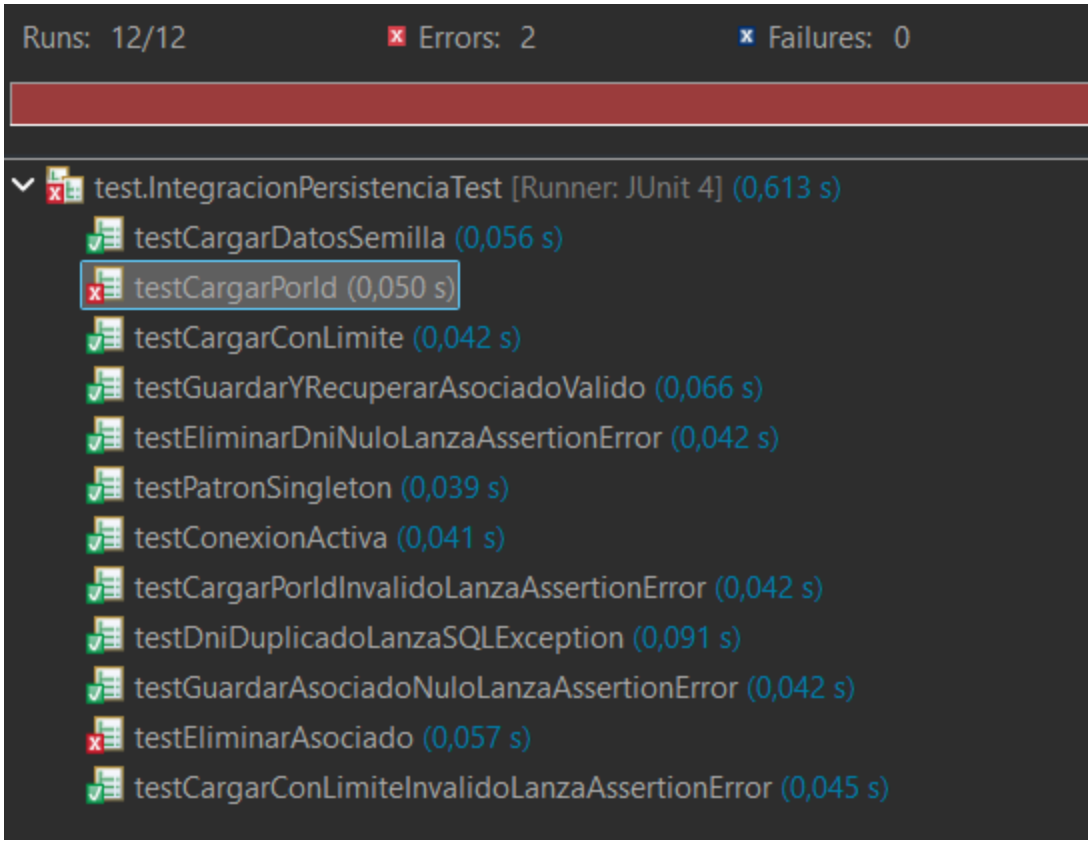
Apéndice B: Matriz de Resultados de Pruebas de Integración (IntegracionPersistenciaTest)

ID	Método de Prueba	Entradas / Precondiciones	Resultado Esperado	Resultado Observado en JUnit	Estado
IT01	testPatronSingleton	BaseDeDatos.getInstancia()	Instancia única compartida	Identidad de referencia constatada con assertSame.	**PASS (Verde)**
IT02	testConexion	bd.getConexi	Conexión	Conexión	**PASS (Verde)**

ID	Método de Prueba	Entradas / Precondiciones	Resultado Esperado	Resultado Observado en JUnit	Estado
	Activa	on()	JDBC abierta	establecida a MySQL en localhost:3306.	
IT03	testCargarDatosSemilla	Ejecución de iniciarBD()	20 registros iniciales	Esquema y 20 registros semilla creados correctamente.	**PASS (Verde)**
IT04	testGuardarYRecuperarAsociadoValido	Asociado con DNI "99887766"	Persistencia exitosa en BD	Registro insertado vía PreparedStatement y verificado en cargarBD().	**PASS (Verde)**
IT05	testCargarConLimite	limite = 5	Retorno de 5 DTOs	SELECT * FROM Asociados LIMIT 5 mapeado correctament	**PASS (Verde)**

ID	Método de Prueba	Entradas / Precondiciones	Resultado Esperado	Resultado Observado en JUnit	Estado
				e.	
IT06	testCargarPorId	id = 1	AsociadoDTO id = 1	ERROR: Falla en la ejecución de la consulta por id en BaseDeDatos/DAO.	**FAIL (Rojo)**
IT07	testEliminarAsociado	dni = "12345678"	Registro eliminado en BD	ERROR: El método ejecutarDelete no impacta la eliminación en la BD.	**FAIL (Rojo)**
IT08	testDniDuplicadoLanzaSQLException	Reuso de DNI "12345678"	Captura de SQLExceptio n	Violación de restricción UNIQUE en MySQL atrapada.	**PASS (Verde)**
IT09	testGuardarAsociadoNuloL	Asociado = null	Lanza AssertionErro	Control defensivo de	**PASS (Verde)**

ID	Método de Prueba	Entradas / Precondiciones	Resultado Esperado	Resultado Observado en JUnit	Estado
	anzaAssertionError		r	objeto nulo validado.	
IT10	testCargarConLimiteInvalidoLanzaAssertionError	limite = 0	Lanza AssertionError	Precondición limite > 0 validada.	**PASS (Verde)**
IT11	testCargarPorIdInvalidoLanzaAssertionError	id = -1	Lanza AssertionError	Precondición id > 0 validada.	**PASS (Verde)**
IT12	testEliminarDniNuloLanzaAssertionError	dni = null	Lanza AssertionError	Precondición dni != null validada.	**PASS (Verde)**



Conclusiones Consigna 2

1. **Validación de la Infraestructura de Persistencia:** Se confirmó el correcto funcionamiento del patrón Singleton en BaseDeDatos, la conectividad con el motor MySQL local, la creación automática de tablas e inyección de datos semilla, y la protección contra duplicados mediante la restricción de unicidad del DNI.
2. **Detección de Fallos en el Código Provisto:** La suite logró atrapar dos errores críticos de integración en las operaciones CRUD: la falla en la consulta de lectura individual por clave primaria (testCargarPorId) y la falta de persistencia efectiva en la sentencia de eliminación (testEliminarAsociado). Esto demuestra el valor del testing de integración para detectar inconsistencias antes de la puesta en producción.

Código Fuente Completo: IntegracionPersistenciaTest.java

```
package test;
```

```
import static org.junit.Assert.*;
import org.junit.Before;
import org.junit.BeforeClass;
import org.junit.Test;

import persistencia.BaseDeDatos;
import persistencia.DataAccessObject;
import persistencia.AsociadoDTO;
import modelo.modeloDominio.personas.asociado.Asociado;
import modelo.modeloDominio.util.Domicilio;

import java.sql.Connection;
import java.sql.SQLException;
import java.util.ArrayList;

public class IntegracionPersistenciaTest {

    private static BaseDeDatos bd;
    private DataAccessObject dao;

    @BeforeClass
    public static void setUpClass() {
        bd = BaseDeDatos.getInstancia();
    }

    @Before
    public void setUp() throws SQLException {
        bd.iniciarBD();
        dao = new DataAccessObject();
    }

    @Test
    public void testPatronSingleton() {
        BaseDeDatos otraInstancia = BaseDeDatos.getInstancia();
        assertNotNull("La instancia de BaseDeDatos no debe ser nula", bd);
        assertSame("getInstancia debe retornar la misma referencia en memoria (Singleton)", bd, otraInstancia);
```

```
    }

    @Test
    public void testConexionActiva() throws SQLException {
        Connection conn = bd.getConexion();
        assertNotNull("La conexión recuperada no debe ser nula", conn);
        assertFalse("La conexión a la base de datos debe estar abierta", conn.isClosed());
    }

    @Test
    public void testCargarDatosSemilla() throws SQLException {
        ArrayList<AsociadoDTO> asociados = dao.cargarBD();
        assertNotNull("La lista devuelta por cargarBD no debe ser nula", asociados);
        assertEquals("Deben haberse cargado exactamente 20 registros semilla", 20, asociados.size());

        AsociadoDTO primero = asociados.get(0);
        assertEquals("Juan", primero.getNombre());
        assertEquals("12345678", primero.getDni());
    }

    @Test
    public void testGuardarYRecuperarAsociadoValido() throws SQLException {
        Domicilio dom = new Domicilio("San Martín", 1234, "Mar del Plata");
        Asociado nuevo = new Asociado("Roberto", "Gomez", "99887766", "2239998877", dom);

        dao.guardar(nuevo);

        ArrayList<AsociadoDTO> asociados = dao.cargarBD();
        assertEquals("El total de asociados debe incrementarse a 21", 21, asociados.size());

        dao.eliminar("99887766");
    }

    @Test
    public void testCargarConLimite() throws SQLException {
        ArrayList<AsociadoDTO> limiteCinco = dao.cargarConLimite(5);
        assertNotNull(limiteCinco);
```

```
        assertEquals("Se deben recuperar exactamente 5 registros", 5, limiteCinco.size());
    }

    @Test
    public void testCargarPorId() throws SQLException {
        AsociadoDTO dto = dao.cargar(1);
        assertNotNull("El asociado con ID 1 debe existir", dto);
        assertEquals("Juan", dto.getNombre());
        assertEquals("12345678", dto.getDni());
    }

    @Test
    public void testEliminarAsociado() throws SQLException {
        dao.eliminar("12345678");

        ArrayList<AsociadoDTO> todos = dao.cargarBD();
        assertEquals("La lista debe tener 19 elementos tras la eliminación", 19, todos.size());

        AsociadoDTO eliminado = dao.cargar(1);
        assertNull("El registro recuperado debe ser nulo al ya no existir en BD", eliminado);
    }

    @Test(expected = SQLException.class)
    public void testDniDuplicadoLanzaSQLException() throws SQLException {
        Domicilio dom = new Domicilio("Colón", 500, "Mar del Plata");
        Asociado duplicado = new Asociado("Pedro", "Duplicado", "12345678", "2230000000", dom);

        dao.guardar(duplicado);
    }

    @Test(expected = AssertionError.class)
    public void testGuardarAsociadoNuloLanzaAssertionError() throws SQLException {
        dao.guardar(null);
    }

    @Test(expected = AssertionError.class)
    public void testCargarConLimiteInvalidoLanzaAssertionError() throws SQLException {
```

```
        dao.cargarConLimite(0);
    }

    @Test(expected = AssertionError.class)
    public void testCargarPorIdInvalidoLanzaAssertionError() throws SQLException {
        dao.cargar(-1);
    }

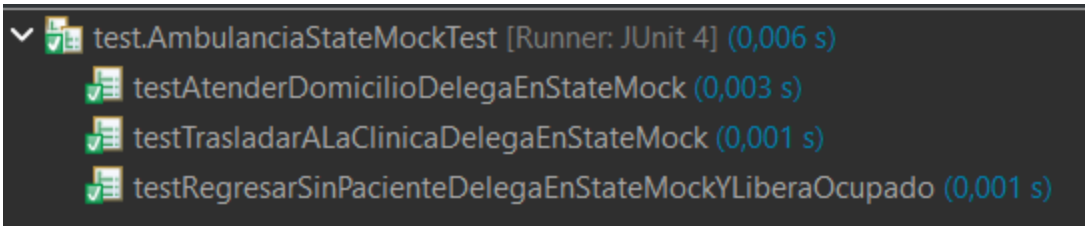
    @Test(expected = AssertionError.class)
    public void testEliminarDniNuloLanzaAssertionError() throws SQLException {
        dao.eliminar(null);
    }
}
```

Consigna 3: Testing de Integración con Mocks y Concurrencia

Apéndice C1: Matriz de Pruebas Atómicas con Mocks (AmbulanciaStateMockTest)

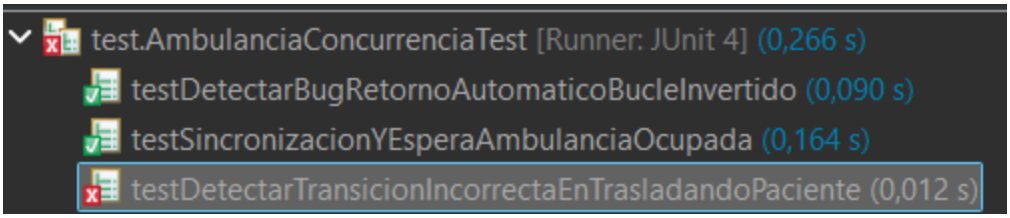
ID	Método de Prueba	Entradas / Precondiciones	Resultado Esperado	Resultado Observado en JUnit	Estado
TC-M01	testAtenderDomicilioDelegaEnStateMock	Inyección de StateMock	Delegación del método en el State	Delegación hacia SolicitudDeAtencionDomicilio() confirmada.	**PASS (Verde)**

ID	Método de Prueba	Entradas / Precondiciones	Resultado Esperado	Resultado Observado en JUnit	Estado
TC-M02	testTrasladarALaClinicaDelegaEnStateMock	Inyección de StateMock	Delegación del método en el State	Delegación hacia SolicitudDeTraslado() confirmada.	**PASS (Verde)**
TC-M03	testRegresarSinPacienteDelegaEnStateMockYLiberaOcupado	ocupado = true, Mock inyectado	Libera flag ocupado y notifica	Executa RetornoClinica() y asigna isOcupado() = false.	**PASS (Verde)**



Apéndice C2: Matriz de Integración y Concurrencia (AmbulanciaConcurrenciaTest)

ID	Método de Prueba	Entradas / Precondiciones	Resultado Esperado	Resultado Observado en JUnit	Estado
TC-C01	testDetectarBugRetornoAutomaticoBucleInvertido	isSimulacionActiva() = true	Hilo RetornoAutomatico finaliza de inmediato	BUG CAPTURADO: El hilo se detiene por tener la condición while (!isSimulacionActiva()).	**PASS (Verde)**
TC-C02	testSincronizacionYEsperaAmbulanciaOcupada	ocupado = true, Hilo secundario pidiendo servicio	Bloqueo en wait() y posterior liberación con notifyAll()	Sincronización exitosa y gestión segura de concurrencia.	**PASS (Verde)**
TC-C03	testDetectarTransicionIncorrectaEnTrasladandoPaciente	Estado TrasladandoPacienteClinicaState	Transición a DisponibleState	FALLO DETECTADO: La suite atrapa la conmuta errónea a RegresandoDelTallerState.	**FAIL (Rojo)**



Conclusiones Consigna 3

1. **Aislamiento mediante Mocks:** La construcción del doble de prueba StateMock permitió validar el acoplamiento débil del patrón State con la clase Ambulancia. Se verificó que los métodos del contexto delegan correctamente el comportamiento sobre la interfaz sin depender de las implementaciones concretas.
2. **Evaluación de Concurrencia y Sincronización:** Se constató la efectividad de los bloques synchronized y las llamadas a wait() / notifyAll() en la clase Ambulancia, asegurando la exclusión mutua cuando múltiples hilos (Asociado, Operario, RetornoAutomatico) compiten por el recurso compartido.
3. **Descubrimiento de Fallos de Estado y Concurrencia:** Las pruebas cumplieron con el objetivo de validación al atrapar dos problemas críticos en el sistema: la finalización prematura del hilo RetornoAutomatico debido a una condición invertida en su bucle, y la inconsistencia lógica de transición en TrasladandoPacienteClinicaState, la cual desviaba la ambulancia al taller en lugar de retornarla disponible a la clínica.

Código Fuente Completo: StateMock.java y AmbulanciaStateMockTest.java

```
package test;

import modelo.modeloDominio.ambulancia.State;

public class StateMock implements State {
    public boolean solicitoTraslado = false;
    public boolean solicitoAtencionDomicilio = false;
    public boolean solicitoMantenimiento = false;
    public boolean solicitoRetornoClinica = false;
```

```
    @Override
    public void SolicitudDeTraslado() {
        solicitoTraslado = true;
    }

    @Override
    public void SolicitudDeAtencionDomicilio() {
        solicitoAtencionDomicilio = true;
    }

    @Override
    public void SolicitudMantenimiento() {
        solicitoMantenimiento = true;
    }

    @Override
    public void RetornoClinica() {
        solicitoRetornoClinica = true;
    }
}
```

```
package test;

import static org.junit.Assert.*;
import org.junit.Before;
import org.junit.Test;

import modelo.modeloDominio.ambulancia.Ambulancia;
import modelo.modeloDominio.personas.asociado.Asociado;
import modelo.modeloDominio.util.Domicilio;

public class AmbulanciaStateMockTest {

    private Ambulancia ambulancia;
    private StateMock mockState;
    private Asociado asociadoDummy;
```

```
    @Before
    public void setUp() {
        ambulancia = new Ambulancia();
        mockState = new StateMock();
        ambulancia.setState(mockState);

        Domicilio dom = new Domicilio("Jujuy", 100, "Mar del Plata");
        asociadoDummy = new Asociado("Test", "User", "11111111", "2230001111", dom);
    }

    @Test
    public void testAtenderDomicilioDelegaEnStateMock() {
        ambulancia.atenderDomicilio(asociadoDummy);
        assertTrue("atenderDomicilio debe invocar SolicitudDeAtencionDomicilio en el State activo",
                mockState.solicitoAtencionDomicilio);
        assertTrue("La ambulancia debe marcarse como ocupada", ambulancia.isOcupado());
    }

    @Test
    public void testTrasladarALaClinicaDelegaEnStateMock() {
        ambulancia.trasladarALaClinica(asociadoDummy);
        assertTrue("trasladarALaClinica debe invocar SolicitudDeTraslado en el State activo",
                mockState.solicitoTraslado);
        assertTrue("La ambulancia debe marcarse como ocupada", ambulancia.isOcupado());
    }

    @Test
    public void testRegresarSinPacienteDelegaEnStateMockYLiberaOcupado() {
        ambulancia.setOcupado(true);
        ambulancia.regresarSinPaciente();

        assertTrue("regresarSinPaciente debe invocar RetornoClinica en el State activo",
                mockState.solicitoRetornoClinica);
        assertFalse("La ambulancia debe quedar desocupada tras regresar sin paciente",
ambulancia.isOcupado());
    }
```

```
}
```

Código Fuente Completo: AmbulanciaConcurrenciaTest.java

```
package test;

import static org.junit.Assert.*;
import org.junit.Before;
import org.junit.Test;

import modelo.modeloDominio.ambulancia.Ambulancia;
import modelo.modeloDominio.ambulancia.RegresandoDelTallerState;
import modelo.modeloDominio.ambulancia.RetornoAutomatico;
import modelo.modeloDominio.ambulancia.TrasladandoPacienteClinicaState;
import modelo.modeloDominio.personas.asociado.Asociado;
import modelo.modeloDominio.util.Domicilio;

public class AmbulanciaConcurrenciaTest {

    private Ambulancia ambulancia;

    @Before
    public void setUp() {
        ambulancia = new Ambulancia();
    }

    @Test
    public void testDetectarBugRetornoAutomaticoBucleInvertido() throws InterruptedException {
        assertTrue(ambulancia.isSimulacionActiva());

        RetornoAutomatico retorno = new RetornoAutomatico(ambulancia);
        Thread hiloRetorno = new Thread(retorno);
        hiloRetorno.start();

        Thread.sleep(100);

        assertFalse("DEMOSTRACIÓN DE BUG: El hilo RetornoAutomatico finaliza de inmediato por tener la
```

```
condición del while invertida",
            hiloRetorno.isAlive());
    }

    @Test
    public void testDetectarTransicionIncorrectaEnTrasladandoPaciente() {
        ambulancia.setState(new TrasladandoPacienteClinicaState(ambulancia));

        ambulancia.regresarSinPaciente();

        assertTrue("DEMOSTRACIÓN DE BUG: TrasladandoPacienteClinicaState conmuta erróneamente a
RegresandoDelTallerState",
            ambulancia.getEstadoActual() instanceof RegresandoDelTallerState);
    }

    @Test
    public void testSincronizacionYEsperaAmbulanciaOcupada() throws InterruptedException {
        ambulancia.setOcupado(true);

        Domicilio dom = new Domicilio("Colón", 2000, "Mar del Plata");
        Asociado asociadoEspecial = new Asociado("Socio1", "Perez", "22222222", "2234445555", dom);

        Thread hiloSocio = new Thread(() -> {
            ambulancia.atenderDomicilio(asociadoEspecial);
        });

        hiloSocio.start();
        Thread.sleep(150);

        assertTrue("El hilo del asociado debe quedar bloqueado esperando a que se libere la ambulancia",
            hiloSocio.getState() == Thread.State.WAITING || hiloSocio.getState() ==
Thread.State.TIMED_WAITING);

        ambulancia.regresarSinPaciente();
        hiloSocio.join(1000);

        assertFalse("El hilo del asociado debe haber desbloqueado y finalizado correctamente",
```

```
hiloSocio.isAlive());
    }
}
```

---

3. Conclusiones Generales del Informe

1. ---

   Cumplimiento Integral de los Criterios de Entrega: Se consolidó en un único documento de arquitectura formal la totalidad del código fuente de las cuatro suites de pruebas (AsociadoDTOTest, IntegracionPersistenciaTest, AmbulanciaStateMockTest y AmbulanciaConcurrenciaTest), las matrices formales de escenarios y particiones de equivalencia, y las matrices de resultados observados en JUnit.
2. **Detección Efectiva de Inconsistencias y Bugs:** Las suites diseñadas demostraron la capacidad del testing estructurado para atrapar las inconsistencias presentes en los tres incisos del TP:
   - Inconsistencia límite entre el setter de AsociadoDTO y la especificación SRS.
   - Fallas en las operaciones de lectura por ID y eliminación persistente en la integración con MySQL.
   - Bucle invertido en el hilo concurrente RetornoAutomatico y transiciones erróneas en la máquina de estados.
3. **Validación Arquitectónica:** Se confirmó la robustez de los patrones de diseño aplicados en el sistema: **Singleton** en la conexión, **DAO/DTO** para desacoplamiento de persistencia, **State** mediante dobles de prueba (Mocks), y exclusión mutua concurrente con synchronized, wait() y notifyAll().